Risk Assessment of Insurence Policies using Machine Learning Techniques

Lukáš Trstenský, Bartosz Kochanski and Maia Ghionea

https://github.com/SAKULAK/ITU-MachineLearning-FinalProject-Couriers

Contents

1 Introduction **2**
- 1.1 Machine Learning Methods 2

2 Data **2**
- 2.1 Data Format 2
- 2.2 Data Cleaning 2
- 2.3 Visualization 3

3 Features **3**
- 3.1 Claims Risk 3
- 3.2 PCA & Clustering 3

4 M1 - Decision Tree **3**
- 4.1 Structure 3
- 4.2 Finding Splits 3
  - 4.2.1 Recursive 3
  - 4.2.2 Priority Queue 3
- 4.3 Stopping Parameters 3

5 M2 - Feed-Forward Neural Network **3**
- 5.1 The Structure 3
- 5.2 The Flow 4
- 5.3 Initialization 4
- 5.4 Forward Pass 4
  - 5.4.1 Activation Functions . . . 4
- 5.5 Backpropagation 4
- 5.6 Updating Parameters 4
- 5.7 Non-Essential Features 4
  - 5.7.1 Early Stopping 5
  - 5.7.2 Mini-Batch Gradient Descent 5
  - 5.7.3 Weight Decay 5

6 Correctness Testing **5**
- 6.1 Comparing Against a Dummy Regressor 5
- 6.2 Simple Function Test 5
  - 6.2.1 Continuous Variables . . . 5
  - 6.2.2 Categorical Variables . . . 5

7 M3 - Ensemble **5**

8 Results **5**

9 Limitations **5**

10 Conclusion **5**
- 10.1 Improvments 5

1 Introduction

This paper aims to explore methods to model the claims risk of insurance policies using various machine learning methods as well as discuss their implementation. We will explore and clean the data, then define the claims risk based on the data available in the dataset.

1.1 Machine Learning Methods

In this paper we will use custom implementation of a Feed-Forward Neural Network Regressor and custom implementation of a Decision Tree Regressor using only Python numerical libraries, e.g. Numpy . Furthermore, we will use an ensemble method using a combination of **TBD**

2 Data

The dataset used in the scopes of this project is the French Motor Third-Part Liability (`freMTPL2freq`) dataset from the `CASdatasets` (Dutang et al., 2024, p. 71) R package. It contains numerical and categorical information on the insurance claims of drivers for their cars spanning multiple regions, details about the cars themselves, such as age, gas type or brand, the drivers, and even the regions where the cars are driven, which more or less impact the risk of a claim being made.

2.1 Data Format

For each unique policy in our dataset, there is a number of claims made within the policy, its duration, then subsequent information on the driver and car. For a clearer overview of the set see Table 1 for the names and corresponding descriptions of the data columns.

Table 1: Data columns and descriptions.

Name	Description
IDpol	Unique policy ID.
ClaimNb	Number of claims made in policy.
Exposure	Duration of policy in years.
Area	Density category of driver's residence from "A" for rural area to "F" for urban centre.
VehPower	Power category of car.
VehAge	Age of car.
DrivAge	Age of driver.
BonusMalus	Bonus/malus, between 50 and 350: < 100 means bonus, > 100 means malus.
VehBrand	The car brand divided in the following groups: A- Renaut Nissan and Citroen, B- Volkswagen, Audi, Skoda and Seat, C- Opel, General Motors and Ford, D- Fiat, E- Mercedes Chryslerand BMW, F- Japanese (except Nissan) and Korean, G- other.
VehGas	Type of gas that the vehicle uses.
Density	Number of inhabitants per km^2 in the city the driver of the car lives in.
Region	The policy region in France (based on the 1970-2015 classification).

2.2 Data Cleaning

A conservative approach was taken in the scope of data cleaning, by minimally modify the data. The material did not present any obvious signs of subverting our assumptions, except for the following:

1. Exposure values should lie in the range between 0 and 1; since the data was collected over the span of one year, we assume that any value above 1 is a mistake in the collection and therefore any value over this range will be reduced to 1, the maximum value permitted.
2. Any data points where $VehAge > 30$ are excluded. This is the cut-off mark at which vehicles become classified as historic cars and will in turn be treated differently by car insurance companies.

Despite the rest of the data not presenting any faults, the information left is narrowed down to solely the figures utilized by the models. Namely, the following columns have been dropped: (**EDIT THESE IF CHANGES**)

a. `IDpol` - not a feature

b. `Area` - categorical counterpart of `Density`

c. `VehBrand` - Vehicle brand alone does not indicate risk of policy since a singular brand has many models of vehicle, some of which might attract more risky drivers, however we believe that the risk evens out along the whole lineup of cars from a brand, well in this case group of brands.

d. `Region` - Another indicator of Density.

2.3 Visualization

As seen in Figure 3 the dataset is heavily biased towards 0 claims. This can pose a significant issue by cluttering the feature space with noise, especially if the differences in feature values for data points with higher numbers of claims and the data points with 0 number of claims are small.

Figure 1: Histogram of the number of claims.

Figure 2: This is an interesting visualization of the data.

3 Features

So after dropping these we are left with our feature columns: `VehPower`, `VehAge`, `DrivAge`, `BonusMalus`, `VehGas`, `Density`
and the columns on which we base our `Risk` calculations: `ClaimNb` and `Exposure`.

3.1 Claims Risk

How do we calculate it, why (idk)
Very imbalanced Figure 3

Figure 3: Histogram of Risk values

3.2 PCA & Clustering

4 M1 - Decision Tree

Simple description of DT, probably also mention the way prediction values are assigned here, because I think it's too short to make a section of it, but if you think otherwise do it that way

Figure 4: PCA with Risk as color — Clustering on PCA data.

4.1 Structure

4.2 Finding Splits

4.2.1 Recursive

4.2.2 Priority Queue

4.3 Stopping Parameters

Why, what problems they solve

Maybe an enumerate of stopping params with simple description of how they work, if that fits, if not use subsubsection for each stopping parameter

a. **Max Depth**
This is the description?

b. **Minimum Samples to Split** -

c. **Minimum Samples at Leaf** -

d. **Maximum number of Leaves** -

e. **Minimum Impurity Decrease** -

5 M2 - Feed-Forward Neural Network

A Feed-Forward Neural Network (*FFNN*) is our second custom implemented regression method. In this section we explain the basics of how a FFNN works along with how an implemention might look.

5.1 The Structure

A FFNN is a collection of nodes organized in layers. First, the input layer. The input layer number of nodes is equal to the number of features in the dataset. Second, the hidden layers. There can be an arbitrary number of hidden layers and of nodes per hidden layer. Generally, the more hidden layers or nodes per layer the more complex the Neural Network becomes, which makes it more prone to overfitting and more computationally heavy to train and run predictions on. Lastly, the output layer. The number of nodes in the output layer depends on the task, but for regression there is just one node, which give the value predicted value for the data points.

Figure 5: Structure of a FFNN

5.2 The Flow

A FFNN at its core follows a very simple flow, see Figure 6, for each iteration or epoch data is run through the network to gain initial predictions, a cost function is calculated, weights are updated based on their partial derivate of the cost function.

Figure 6: Flowchart of a FFNN

5.3 Initialization

We initialize an array of weights (W_{ij}) and an array of biases (b_j) for each pair of subsequent layers. The shape of W_ij is the number of nodes in the previous layer (i) by the number of nodes in the current layer (j) which is also the shape of the bias array.

To fill the arrays with initial values we use the He Initialization (He et al., 2015) by drawing from a zero-mean normal distribution with standard deviation of $\sqrt{2/n_i}$ which is considered to be among the best initializations for shallow ReLU based FFNNs.

5.4 Forward Pass

During the Forward Pass, the data, as the name suggests propagates forward through the network. At each layer, except the input layer, the activations of the previous layer (a_i) are weighed by the weights and have the bias added to them:

$$z_j = a_i W_{ij} + b_j$$

Then they are passed through the activation function to become the activation of the current layer, which are then passed to the next layer.

5.4.1 Activation Functions

There are many activation functions that one might choose depending on the task. Usually you use two different activation functions, one for the hidden layers and a different one for the output layer. For hidden layers we use the widely used activation function, the Rectified Linear Unit (*ReLU*) function defined as $ReLU(x) = max(0, x)$. For the output layer of regression FFNNs a simple linear function is used $f(x) = x$

5.5 Backpropagation

Backpropagation is the step of the training process, which allows the model to learn. By utilizing gradient descent in the cost function landscape the weights can be updated in the direction and magnitude which results in the updated weights producing a less costly prediction. However, to be able to use gradient descent we need the partial derivatives of the cost function relative to each weight. This can be done fully symbolically, which for large networks is infeasible, but because of the chain rule, this can be simplified into a recursive process in which the derivative of the cost function relative to the weights of the output layer (j) is computed:

$$\frac{\partial C}{\partial w_{ij}} = \frac{\partial z_j}{\partial w_{ij}}(\frac{\partial C}{\partial z_j}) = \frac{\partial z_j}{\partial w_{ij}}(\frac{\partial a_j}{\partial z_j}\frac{\partial C}{\partial a_j}) =$$

$$a_i(\Phi'(z_j)C'(a_j))$$

The error term of the partial derivate for output layer for $\Phi = f(z_j) = z_j$ and $C = MSE = (y_{pred} - y_{true})^2/n$ is:

$$\frac{\partial C}{\partial z_j} = (1\frac{2(a - y_{true})}{n})$$

Then the effects of the weighted sum of the previous layer, in this case the last hidden layer (i), on the cost function is calculated:

$$\frac{\partial C}{\partial w_{ui}} = \frac{\partial z_i}{\partial w_{ui}}[\frac{\partial C}{\partial z_i}] = \frac{\partial z_i}{\partial w_{ui}}[\frac{\partial a_i}{\partial z_i}\frac{\partial z_j}{\partial a_i}(\frac{\partial C}{\partial z_j})] =$$

$$a_u[\Phi'(z_i)w_{ij}(\frac{\partial C}{\partial z_j})]$$

where $\Phi = ReLU$ then $\Phi' = 1\{z \geq 0\}$.
This holds true for any layer going backwards from the output layer. Its partial derivative has same terms, only depending on its own parameters and the error term of the next layer ($\frac{\partial C}{\partial z_j}$).
The same derivation is done for the bias term, however to maintain its correct dimensions the contributions of each data point for a singular bias term are averaged and $\frac{\partial C}{\partial b_i} = \frac{\partial z_i}{\partial b_i}$[error term] $= 1$[error term].

5.6 Updating Parameters

5.7 Non-Essential Features

For better flexibility and performance of the model regularization, early stopping and mini-batch gradient descent is implemented.

5.7.1 Early Stopping

The training data is randomly split into a training and a validation set with the size of the validation set being 10% of the initial data.
After each epoch the cost is calculated on the validation set, if the cost increases for n consecutive epochs training is aborted and weights are set to be the weights from the best performing epoch. This helps to optimize for performance while allowing for lower training time.

5.7.2 Mini-Batch Gradient Descent

Not sure if we should leave this in.

5.7.3 Weight Decay

6 Correctness Testing

To validate our implementation of the Decision Tree and the Feed-Forward Neural Network we performed correctness testing.

6.1 Comparing Against a Dummy Regressor

6.2 Simple Function Test

6.2.1 Continuous Variables

6.2.2 Categorical Variables

7 M3 - Ensemble

Idk let's see what we do

8 Results

Probably some table of results of each method and how and why we evaluated like we did.

9 Limitations

Probably could have somehow cleaned the data better to get better separation of Risk values and would like claim amounts.

10 Conclusion

Hopefully something good. Summarize the results of each method, include running times say which one worked best as a combination of time and performance

10.1 Improvments

everything

References

[Dutang et al., 2024] Dutang, C., Charpentier, A., Gallic, E., and Siharath, J. (2024). CASdatasets: Insurance datasets. `https://cas.uqam.ca/pub/web/CASdatasets-manual.pdf`. R package version 1.2-0.

[He et al., 2015] He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification.

[Kingma and Ba, 2017] Kingma, D. P. and Ba, J. (2017). Adam: A method for stochastic optimization.